

Singleton

Lập trình hướng đối tượng

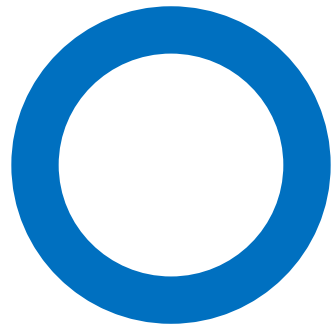
Hồ Tuấn Thanh

htthanh@fit.hcmus.edu.vn

Nội dung

- ☐ Thành phần tĩnh
- ☐ Thuộc tính tĩnh
- ☐ Hàm tĩnh
- ☐ Singleton

Thành phần tĩnh



Giới thiệu

□ Trong một lớp sẽ các thành phần sau:

- Các thuộc tính

- Các hàm

 - Các hàm dựng, hàm hủy

 - Các hàm xử lý tính toán

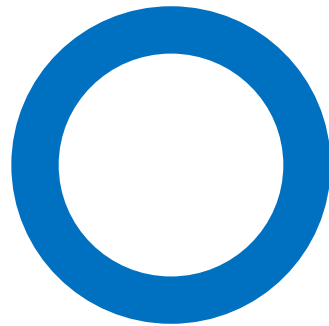
 - Hàm toán tử

 - KHÔNG tính hàm friend***

Giới thiệu

- ❑ Các thành phần (các thuộc tính, các hàm) mà ta đã khai báo và cài đặt trong các tuần trước gọi là ***thành phần của đối tượng***.
- ❑ Hôm nay, ta sẽ học về các ***thành phần*** (các thuộc tính, các hàm) ***của lớp***, hay gọi là ***thành phần tĩnh***.
- ❑ Như vậy, bài học hôm nay là về:
 - Thuộc tính tĩnh
 - Hàm tĩnh
- ❑ Vậy thuộc tính tĩnh, hàm tĩnh khác gì so với thuộc tính, hàm đã học trước đó?

Thuộc tính tĩnh



Giới thiệu

- ❑ ***Thuộc tính tĩnh*** khác thuộc tính đã khai báo ở các tuần trước (thuộc tính của đối tượng) chỗ nào?
 - Khai báo: Thêm từ khóa static ở đằng trước.
 - Phải khai báo lại trước hàm main (chỉ C++ mới phải làm bước này)
 - Sử dụng: ***TenLop::TenThuocTinh***

Giới thiệu

```
class A
{
private:
    // Thuộc tính của đối tượng
    int x;
    // Thuộc tính static
    // Thuộc tính của lớp
    static int t;
public:
    void f()
    {
        cout<<A::t<<endl;
    }
};
```

```
// Khai báo lại thuộc tính static
// trước khi sử dụng
// thường đặt trước hàm main
int A::t;
void main()
{
}
```


Lưu ý

- ❑ Trong ***hàm của đối tượng***, có thể truy xuất ***thuộc tính tĩnh***.
 - Hàm f (***hàm bình thường***) có thể truy xuất thuộc tính t (***thuộc tính tĩnh***)

```
class A
{
private:
    // Thuộc tính của đối tượng
    int x;
    // Thuộc tính static
    // Thuộc tính của lớp
    static int t;
public:
    void f()
    {
        cout<<A::t<<endl;
    }
};
```

DUNG!!!

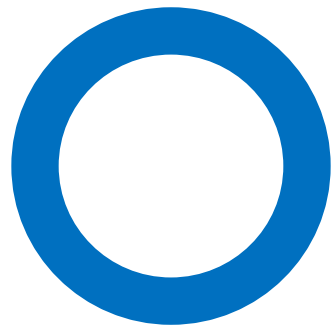
Lưu ý

- ❑ Khi truy cập thuộc tính tĩnh, phải dùng cú pháp ***TenLop::TenThuocTinh***, không được dùng cú pháp ***this->TenThuocTinh***

```
class A
{
private:
    // Thuộc tính của đối tượng
    int x;
    // Thuộc tính static
    // Thuộc tính của lớp
    static int t;
public:
    void f()
    {
        cout<<this->t<<endl;
    }
};
```

SAI!!!

Hàm tĩnh



Giới thiệu

- ❑ **Hàm tĩnh** khác với hàm đã khai báo các tuần trước (hàm của đối tượng) chỗ nào?
 - Khai báo: thêm từ khóa static đằng trước.
 - Sử dụng: ***TenLop::TenHam(CacThamSo);***
- ❑ Có cần khai báo lại trước hàm main như thuộc tính tĩnh không? ➔ KHÔNG!!!

Giới thiệu

```
class A
{
public:
    // Hàm tĩnh
    // Hàm của lớp
    static void p()
    {
        cout<<"Ham p"<<endl;
    }
};

void main()
{
    A::p();
}
```

Lưu ý

- ❑ Khi truy cập hàm tĩnh, phải dùng cú pháp ***TenLop::TenHam***, không được dùng cú pháp ***TenDoiTuong.TenHam***

Lưu ý

```
class A
{
public:
    void f()
    {
        A::p(); DUNG!!!
    }
    // Hàm tĩnh
    // Hàm của lớp
    static void p()
    {
        cout<<"Ham p"<<endl;
    }
};

void main()
{
    A::p(); DUNG!!!
}
```

```
class A
{
public:
    void f()
    {
        this->p(); SAI!!!
    }
    // Hàm tĩnh
    // Hàm của lớp
    static void p()
    {
        cout<<"Ham p"<<endl;
    }
};

void main()
{
    A a1;
    a1.p(); SAI!!!
}
```

Lưu ý

- ❑ **Hàm của đối tượng** có thể truy xuất được **tất cả**, bao gồm: thuộc tính của đối tượng, thuộc tính của lớp (thuộc tính tĩnh), hàm của đối tượng, hàm của lớp (hàm tĩnh)

Lưu ý

```
class A
{
private:
    int x;           // thuộc tính của đối tượng
    static int t;    // thuộc tính của lớp
public:
    void f()         // hàm của đối tượng
    {

    }

    void g()         // hàm của đối tượng
    {
        cout<<this->x<<endl;    // Đúng
        cout<<A::t<<endl;      // Đúng
        this->f();              // Đúng
        A::p();                // Đúng
    }
    static void p() // hàm của lớp
    {
        cout<<"Ham p"<<endl;
    }
};
```

Lưu ý

- ❑ **Hàm của lớp** có thể truy xuất thuộc tính của lớp (thuộc tính tĩnh), hàm của lớp (hàm tĩnh), **KHÔNG THỂ truy xuất thuộc tính của đối tượng và hàm của đối tượng.**

Lưu ý

```
class A
{
private:
    int x;           // thuộc tính của đối tượng
    static int t;    // thuộc tính của lớp
public:
    void f()         // hàm của đối tượng
    {

    }

    void g()         // hàm của đối tượng
    {

    }

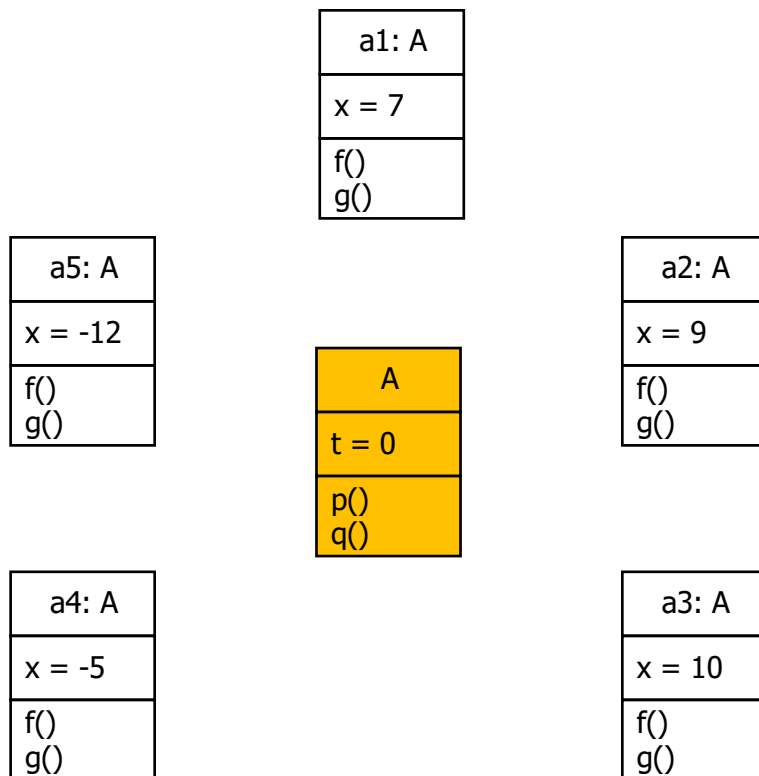
    static void p() // hàm của lớp
    {
        cout<<"Ham p"<<endl;
    }

    static void q() // hàm của lớp
    {
        cout<<this->x<<endl;    // Sai
        cout<<A::t<<endl;      // Đúng
        this->f();              // Sai
        A::p();                 // Đúng
    }
};
```

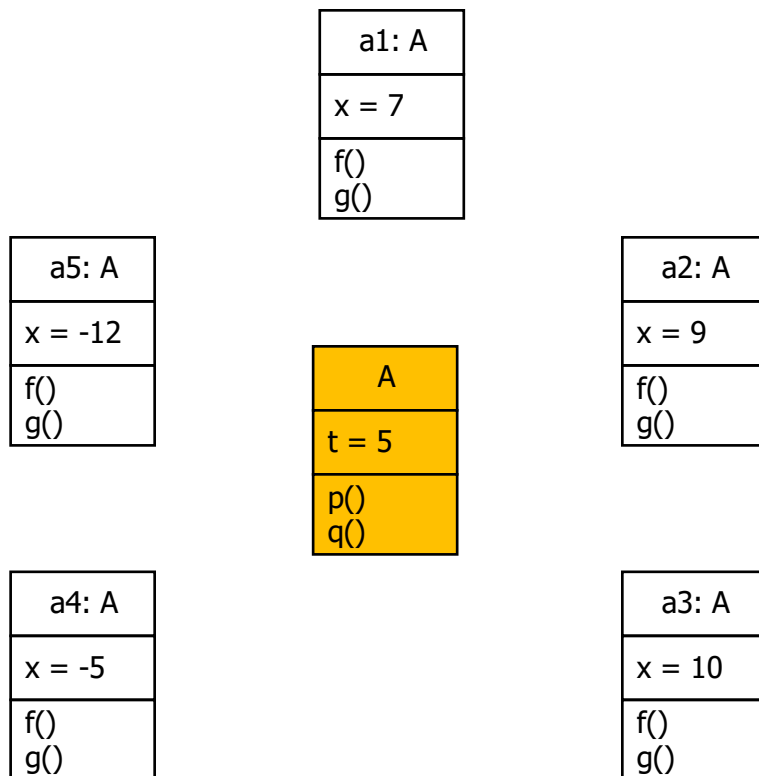
Lưu ý

- ❑ Tại sao lại gọi là:
 - Thuộc tính của đối tượng
 - Hàm của đối tượng
 - Thuộc tính của lớp
 - Hàm của lớp
- ❑ Xem hình minh họa sau

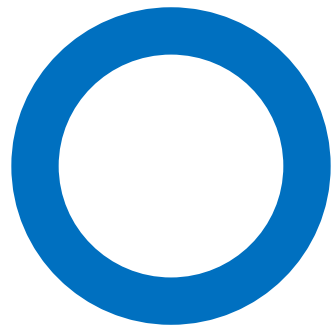
Lưu ý



Lưu ý



Ứng dụng



Ứng dụng

- ❑ **Thuộc tính tĩnh** dùng để khai báo **biến toàn cục** cho một lớp.
- ❑ **Hàm tĩnh** dùng để thay thế hàm của đối tượng khi các thao tác trong hàm **không truy xuất đến các thuộc tính của đối tượng hoặc các hàm của đối tượng**.

VD1: PhanSo

- ❑ Nhu cầu: Đếm số đối tượng PhanSo được tạo ra
- ❑ Giải pháp

Giải pháp 1: Biến toàn cục

- ❑ Khai báo một biến đếm toàn cục.
- ❑ Khi đối tượng PhanSo được tạo ra → Một trong các phương thức thiết lập sẽ được gọi → Trong các phương thức thiết lập, tăng giá trị biến đếm lên 1

```
int soDoiTuongPhanSo=0;
```

```
PhanSo::PhanSo(void)  
{  
    soDoiTuongPhanSo++;  
    tu=0;  
    mau=1;  
}
```

Giải pháp 1: Biến toàn cục

```
PhanSo::PhanSo(int x)
{
    soDoiTuongPhanSo++;
    tu=x;
    mau=1;
}
```

```
PhanSo::PhanSo(int t, int m)
{
    soDoiTuongPhanSo++;
    tu=t;
    mau=m;
}
```

Giải pháp 1: Biến toàn cục

```
PhanSo::PhanSo(const PhanSo& ps)
{
    soDoiTuongPhanSo++;
    tu=ps.tu;
    mau=ps.mau;
}
```

```
PhanSo PhanSo::operator*(const PhanSo& ps)
{
    PhanSo kq;
    kq.tu=tu*ps.tu;
    kq.mau=mau*ps.mau;
    return kq;
}
```

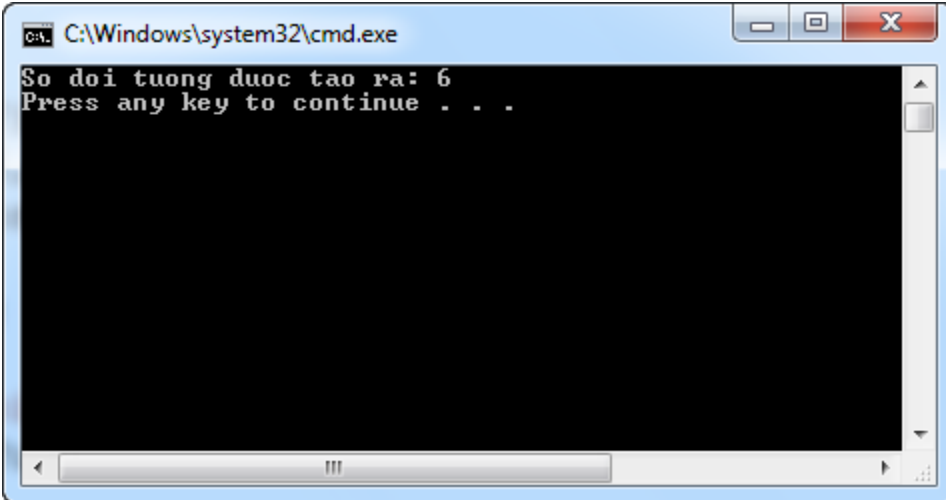
```
ostream& operator<<(ostream& os, const PhanSo& ps)
{
    os<<ps.tu<<"/"<<ps.mau;
    return os;
}
```

Giải pháp 1: Biến toàn cục

```
void main()
{
    PhanSo ps1;
    PhanSo ps2(7);
    PhanSo ps3(6, 9);
    PhanSo ps4(ps1);

    ps4=ps1*ps2;

    cout<<"So doi tuong duoc tao ra: "
         <<soDoiTuongPhanSo<<endl;
}
```



The screenshot shows a Windows command prompt window titled "C:\Windows\system32\cmd.exe". The output of the program is displayed as follows:

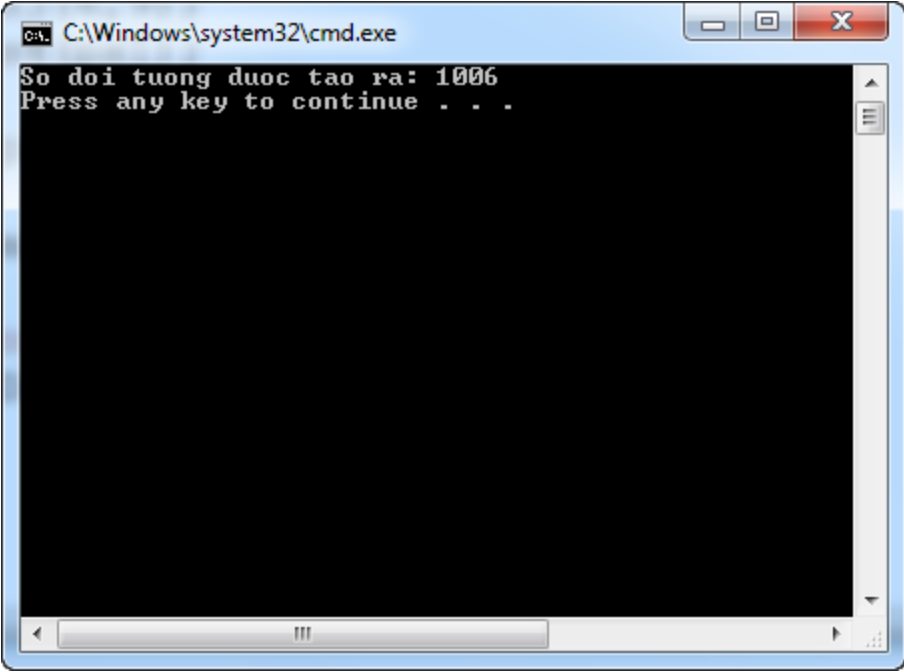
```
So doi tuong duoc tao ra: 6
Press any key to continue . . .
```

Giải pháp 1: Biến toàn cục → Vấn đề

```
void hamPhaHoai()  
{  
    soDoiTuongPhanSo+=1000;  
}
```

```
void main()  
{  
    PhanSo ps1;  
    PhanSo ps2(7);  
    PhanSo ps3(6,9);  
    PhanSo ps4(ps1);  
  
    ps4=ps1*ps2;  
  
    hamPhaHoai();  
  
    cout<<"So doi tuong duoc tao ra: "  
        <<soDoiTuongPhanSo<<endl;  
}
```

Giải pháp 1: Biến toàn cục → Vấn đề



A screenshot of a Windows command prompt window. The title bar shows the path `C:\Windows\system32\cmd.exe`. The window contains the following text:

```
So doi tuong duoc tao ra: 1006  
Press any key to continue . . .
```

The window has a standard Windows interface with a blue title bar, minimize/maximize/close buttons, and a scrollbar on the right side.

Giải pháp 2: Thành phần tĩnh

PhanSo

- ❑ soDoiTuongPhanSo
- ❑ LaySoDoiTuongPhanSo();

ps1: PhanSo

- ❑ tuSo: 0
- ❑ mauSo: 1

ps2: PhanSo

- ❑ tuSo: 7
- ❑ mauSo: 1

ps3: PhanSo

- ❑ tuSo: 6
- ❑ mauSo: 9

ps4: PhanSo

- ❑ tuSo: 0
- ❑ mauSo: 1

Giải pháp 2: Thành phần tĩnh

```
class PhanSo
{
private:
    int tu, mau;
    static int soDoiTuongPhanSo;
public:
    PhanSo(void);
    PhanSo(int x);
    PhanSo(int t, int m);
    PhanSo(const PhanSo& ps);
    static int LaySoDoiTuongPhanSo();
}
```

Thuộc tính tĩnh

Phương thức tĩnh

```
PhanSo::PhanSo(void)
{
    soDoiTuongPhanSo++;
    tu=0;
    mau=1;
}
```

Giải pháp 2: Thành phần tĩnh

```
PhanSo::PhanSo(int x)
{
    PhanSo::soDoiTuongPhanSo++;
    tu=x;
    mau=1;
}
```

```
PhanSo::PhanSo(int t, int m)
{
    soDoiTuongPhanSo++;
    tu=t;
    mau=m;
}
```

Giải pháp 2: Thành phần tĩnh

```
PhanSo::PhanSo(const PhanSo& ps)
{
    soDoiTuongPhanSo++;
    tu=ps.tu;
    mau=ps.mau;
}
```

```
PhanSo PhanSo::operator*(const PhanSo& ps)
{
    PhanSo kq;
    kq.tu=tu*ps.tu;
    kq.mau=mau*ps.mau;
    return kq;
}
```

```
ostream& operator<<(ostream& os, const PhanSo& ps)
{
    os<<ps.tu<<"/"<<ps.mau;
    return os;
}
```

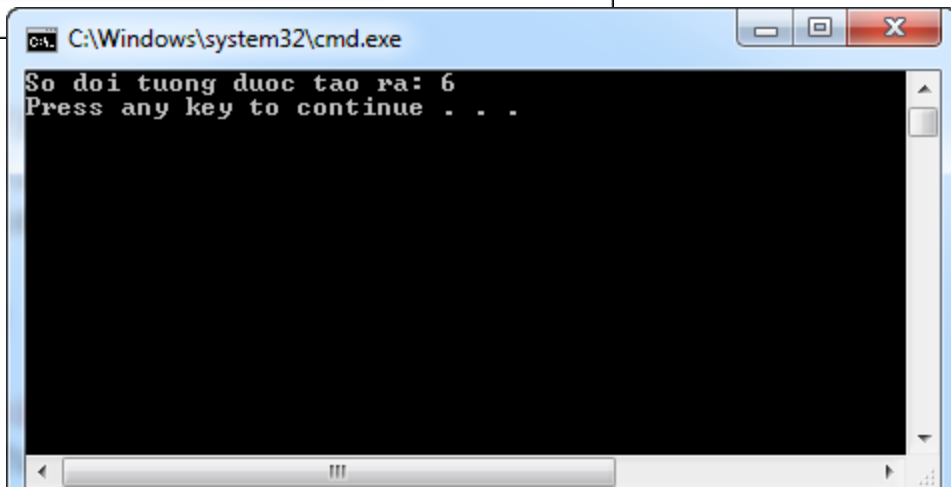
Giải pháp 2: Thành phần tĩnh

```
int PhanSo::soDoiTuongPhanSo=0;

void main()
{
    PhanSo ps1;
    PhanSo ps2(7);
    PhanSo ps3(6,9);
    PhanSo ps4(ps1);

    ps4=ps1*ps2;

    cout<<"So doi tuong duoc tao ra: "
         <<PhanSo::LaySoDoiTuongPhanSo()<<endl;
}
```



The screenshot shows a Windows command prompt window titled "C:\Windows\system32\cmd.exe". The output of the program is displayed as follows:

```
So doi tuong duoc tao ra: 6
Press any key to continue . . .
```

VD2: Hàm UCLN

- ❑ Hàm tìm UCLN không cần truy xuất các thuộc tính (tu, mau) hay các hàm (Cong, Tru, ...) ➔ ta có thể khai báo UCLN là hàm tĩnh.

```
class PhanSo
{
    private:
        int tu;
        int mau;
        static int UCLN(int x, int y);
    public:
        PhanSo Cong(PhanSo p);
        PhanSo Tru(PhanSo p);
        void RutGon();
};
```

VD3: Lớp Math

- ❑ Không sử dụng hàm tĩnh

```
void main()
{
    Math m;
    m.GanX(3.14);
    m.Sin();
    cout<<m.LayKetQua()<<endl;
}
```

```
class Math
{
private:
    float x;
    float kq;
public:
    void Sin()
    {
        kq=sin(x);
    }
    void GanX(float v)
    {
        x=v;
    }
    float LayKetQua()
    {
        return kq;
    }
};
```

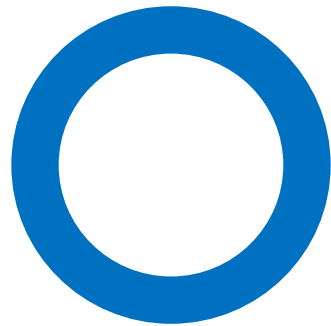
VD3: Lớp Math

❑ Sử dụng hàm tĩnh

```
class Math
{
public:
    static float Sin(float x)
    {
        return sin(x);
    }
};

void main()
{
    cout<<Math::Sin(3.14)<<endl;
}
```

Design pattern



Design pattern

- ❑ Pattern: mẫu, khuôn mẫu
- ❑ Design: thiết kế
- ❑ Chúng ta đang học môn OOP
- ❑ ➔ Design patterns: Các mẫu thiết kế sơ đồ lớp trong lập trình hướng đối tượng.

Design pattern

- ❑ Đó là các giải pháp tối ưu, có thể tái sử dụng cho các vấn đề lập trình ta gặp hàng ngày.
- ❑ Đó ko phải là một class hay một library mà ta include vào hệ thống để sử dụng.
- ❑ Đó là một template chỉ ta cách cài đặt làm sao cho đúng.

Ví dụ 1

- ❑ Tôi cần có một class, mà chỉ có thể tạo 1 đối tượng cho class đó.
 - Cách 1: suy nghĩ vài ngày, tìm ra giải pháp.
 - Cách 2: đơn giản, đã học mẫu Singleton rồi ➔ Lấy ra cài đặt theo đó thôi.

Ví dụ 2

- ❑ Một cái máy gồm nhiều chi tiết. Một chi tiết có thể là chi tiết đơn hoặc chi tiết phức. Chi tiết phức có thể chứa nhiều chi tiết trong đó. Hãy tính tổng trọng lượng của cái máy.
 - Cách 1: suy nghĩ vài tuần sẽ ra giải pháp.
 - Cách 2: hồi trước có học mẫu Composite từ bài tập tin, thư mục rồi (một thư mục có thể chứa nhiều thư mục hoặc tập tin con) ➔ Lấy ra cài đặt tương tự lại thôi.

23 mẫu thiết kế cơ bản

- ❑ Gamma, Erich; Richard Helm, Ralph Johnson, and John Vlissides (1995). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. ISBN 0-201-63361-2.
 - Gang of Four – GoF
- ❑ 23 mẫu thiết kế được chia làm 3 nhóm
 - Creational patterns
 - Structural patterns
 - Behavioral patterns

Creational patterns

- ☐ Abstract Factory
- ☐ Builder
- ☐ Factory Method
- ☐ Prototype
- ☐ Singleton

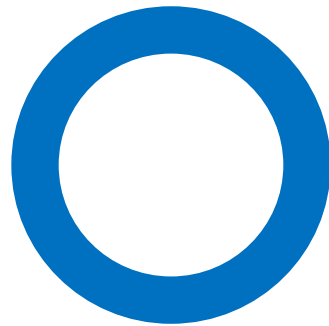
Structural patterns

- ☐ Adapter
- ☐ Bridge
- ☐ Composite
- ☐ Decorator
- ☐ Façade
- ☐ Flyweight
- ☐ Proxy

Behavioral patterns

- ☐ Chain of responsibility
- ☐ Command
- ☐ Interpreter
- ☐ Iterator
- ☐ Mediator
- ☐ Memento
- ☐ Observer
- ☐ State
- ☐ Strategy
- ☐ Template method
- ☐ Visitor

Singleton pattern



Đặt vấn đề

- ❑ Đảm bảo một class chỉ có duy nhất một đối tượng (instance).
- ❑ Có thể truy cập toàn cục.

Ví dụ

❑ VD1: Logger

- Ứng dụng trong quá trình chạy, cần ghi lại nhật kí hoạt động: các lỗi gặp phải, các thao tác quan trọng trong ứng dụng. Trong một ứng dụng thì chỉ có một logger duy nhất.
- <https://www.mkyong.com/logging/log4j-hello-world-example/>

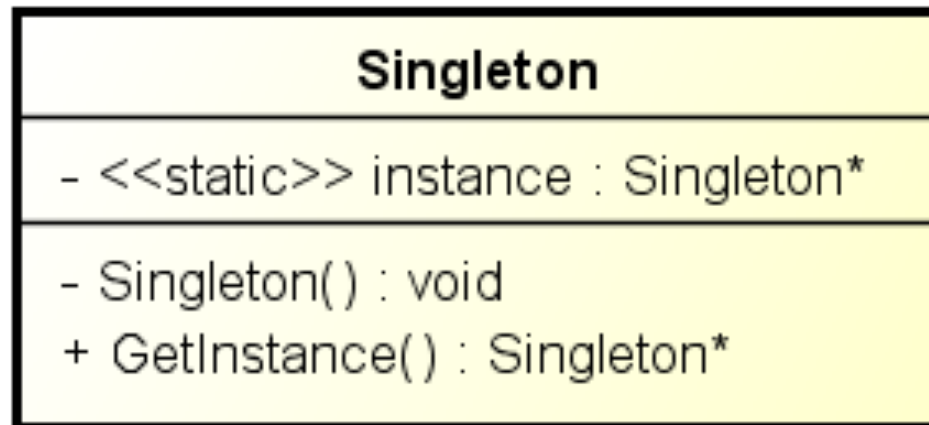
❑ VD2: Configuration

- Mỗi ứng dụng chỉ có một bộ cấu hình duy nhất cho nó. Các thông tin trong bộ cấu hình này cần được từ truy cập toàn cục trong ứng dụng.

Ý tưởng cài đặt

- ❑ Đặt các hàm dựng (constructor) tạo lập dạng private/protected để người dùng không thể tự ý tạo đối tượng mới. Ta cần cài đặt hàm dựng để tránh người dùng gọi hàm dựng mặc định.
- ❑ Tạo sẵn một đối tượng duy nhất ở dạng static để người dùng chỉ được phép sử dụng đối tượng đã có sẵn này. Tạo hàm static để người dùng lấy đối tượng.

Sơ đồ lớp



powered by Astah 

Cài đặt

```
class Singleton{
private:
    Singleton();
    static Singleton *instance;
public:
    static Singleton* getInstance();
};

Singleton::Singleton(){
    cout << "An instance created" << endl;
}

Singleton* Singleton::getInstance(){
    if(instance == NULL)
        instance = new Singleton();
    return instance;
}
```

Singleton vs Static

- ❑ Sách của GoF
- ❑ <http://stackoverflow.com/questions/519520/difference-between-static-class-and-singleton-pattern>

